

WEEK 3: SELLER AND INVENTORY MANAGEMENT SYSTEM

Scenario:

An e-commerce platform may have products supplied by different sellers. To manage these products properly, the database should maintain seller details and keep track of the stock available for each product.

In this week, the **SELLER** and **INVENTORY** tables are used along with the existing **PRODUCT** table. The tables are connected using keys so that seller information, product details, and inventory information can be managed together.

Create Seller and Inventory Tables

Seller Table:

```
SQL> CREATE TABLE SELLER(SELLER_ID VARCHAR2(7), SELLER_NAME VARCHAR2(25),EMAIL VARCHAR2(30),PHONE NUMBER(10), ADDRESS VARCHAR2(100));

Table created.

SQL> DESC SELLER;
Name                               Null?    Type
-----
SELLER_ID                          VARCHAR2(7)
SELLER_NAME                        VARCHAR2(25)
EMAIL                              VARCHAR2(30)
PHONE                              NUMBER(10)
ADDRESS                            VARCHAR2(100)
```

Inventory Table:

```
SQL> CREATE TABLE INVENTORY (
2     INVENTORY_ID VARCHAR2(7),
3     PRODUCT_ID VARCHAR2(7),
4     SELLER_ID VARCHAR2(7),
5     STOCK_QUANTITY NUMBER,
6     STOCK_STATUS VARCHAR2(20),
7     LAST_UPDATED DATE
8 );

Table created.

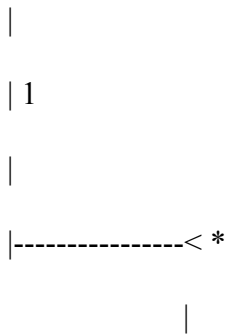
SQL> DESC INVENTORY;
Name                               Null?    Type
-----
INVENTORY_ID                       VARCHAR2(7)
PRODUCT_ID                         VARCHAR2(7)
SELLER_ID                          VARCHAR2(7)
STOCK_QUANTITY                     NUMBER
STOCK_STATUS                       VARCHAR2(20)
LAST_UPDATED                       DATE
```

Establish Relationships :

Seller and Product:

A seller can provide multiple products. The **SELLER_ID** in the **PRODUCT** table is used to connect a product with its seller. The Week 3 document describes this as a one-to-many relationship.

SELLER

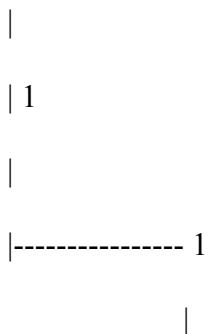


PRODUCT

Product and Inventory:

The inventory table maintains stock information for each product. The Week 3 document specifies the Product–Inventory relationship as one-to-one.

PRODUCT



INVENTORY

Define Keys :

```
SQL> ALTER TABLE SELLER
  2  ADD CONSTRAINT PK_SELLER
  3  PRIMARY KEY (SELLER_ID);
```

Table altered.

```
SQL> ALTER TABLE INVENTORY
  2  ADD CONSTRAINT PK_INVENTORY
  3  PRIMARY KEY (INVENTORY_ID);
```

Table altered.

```
SQL> ALTER TABLE PRODUCT
  2  ADD SELLER_ID VARCHAR2(7);
```

Table altered.

```
SQL> ALTER TABLE PRODUCT
  2  ADD CONSTRAINT FK_PRODUCT_SELLER
  3  FOREIGN KEY (SELLER_ID)
  4  REFERENCES SELLER(SELLER_ID);
```

Table altered.

Apply Constraints :

```
SQL> ALTER TABLE INVENTORY
  2  ADD CONSTRAINT FK_INVENTORY_PRODUCT
  3  FOREIGN KEY (PRODUCT_ID)
  4  REFERENCES PRODUCT(PRODUCT_ID);
```

Table altered.

```
SQL> ALTER TABLE INVENTORY
  2  ADD CONSTRAINT UQ_INVENTORY_PRODUCT
  3  UNIQUE (PRODUCT_ID);
```

Table altered.

```
SQL> ALTER TABLE SELLER
  2  ADD CONSTRAINT UQ_SELLER_EMAIL
  3  UNIQUE (EMAIL);
```

Table altered.

```
SQL> ALTER TABLE SELLER
  2  MODIFY EMAIL CONSTRAINT NN_SELLER_EMAIL NOT NULL;
```

Table altered.

```
SQL> ALTER TABLE INVENTORY
  2  ADD CONSTRAINT CHK_INVENTORY_STOCK
  3  CHECK (STOCK_QUANTITY >= 0);
```

Table altered.

Insert Values:

Seller Table:

```
SQL> INSERT INTO SELLER VALUES('&SELLER_ID','&SELLER_NAME','&EMAIL','&PHONE','&ADDRESS');
Enter value for seller_id: S001
Enter value for seller_name: Tech World
Enter value for email: techworld@gmail.com
Enter value for phone: 9876543210
Enter value for address: Chennai
old 1: INSERT INTO SELLER VALUES('&SELLER_ID','&SELLER_NAME','&EMAIL','&PHONE','&ADDRESS')
new 1: INSERT INTO SELLER VALUES('S001','Tech World','techworld@gmail.com',9876543210,'Chennai')

1 row created.
```

Table

```
SQL> SELECT * FROM SELLER;
```

SELLER_	SELLER_NAME	EMAIL	PHONE	ADDRESS
S001	Tech World	techworld@gmail.com	9876543210	Chennai
S002	Style Hub	stylehub@gmail.com	9876543211	Coimbatore
S003	Book House	bookhouse@gmail.com	9876543212	Madurai
S004	Home Needs	homeneeds@gmail.com	9876543213	Salem
S005	Sport Zone	sportzone@gmail.com	9876543214	Trichy
S006	Gadget Point	gadgetpoint@gmail.com	9876543215	Bangalore
S007	Fashion World	fashionworld@gmail.com	9876543216	Hyderabad
S008	Daily Mart	dailymart@gmail.com	9876543217	Pondicherry
S009	Smart Home	smarthome@gmail.com	9876543218	Bangalore
S010	Fit Life	fitlife@gmail.com	9876543219	Chennai

```
10 rows selected.
```

Inventory Table :

```
SQL> INSERT INTO INVENTORY VALUES('&INVENTORY_ID','&PRODUCT_ID','&SELLER_ID','&STOCK_QUANTITY','&STOCK_STATUS','&LAST_UPDATED');
Enter value for inventory_id: I001
Enter value for product_id: P001
Enter value for seller_id: S001
Enter value for stock_quantity: 50
Enter value for stock_status: Available
Enter value for last_updated: SYSDATE
old 1: INSERT INTO INVENTORY VALUES('&INVENTORY_ID','&PRODUCT_ID','&SELLER_ID','&STOCK_QUANTITY','&STOCK_STATUS','&LAST_UPDATED')
new 1: INSERT INTO INVENTORY VALUES('I001','P001','S001',50,'Available',SYSDATE)

1 row created.
```

Table

```
SQL> SELECT * FROM INVENTORY;
```

INVENTO	PRODUCT	SELLER_	STOCK_QUANTITY	STOCK_STATUS	LAST_UPDA
I001	P001	S001	50	Available	17-SEP-26
I002	P002	S002	30	Available	17-SEP-26
I003	P003	S003	8	Low Stock	17-SEP-26
I004	P004	S004	40	Available	17-SEP-26
I005	P005	S005	15	Available	17-SEP-26
I006	P006	S006	5	Low Stock	17-SEP-26
I007	P007	S007	25	Available	17-SEP-26
I008	P008	S008	0	Out of Stock	17-SEP-26
I009	P009	S009	18	Available	17-SEP-26
I010	P010	S010	3	Low Stock	17-SEP-26

```
10 rows selected.
```

Report 1: Seller-wise Product Report

This report shows the seller name, number of products supplied by the seller, and the total stock available for those products.

```
SQL> SELECT s.SELLER_NAME,  
2         COUNT(p.PRODUCT_ID) AS PRODUCT_COUNT,  
3         SUM(i.STOCK_QUANTITY) AS AVAILABLE_STOCK  
4 FROM SELLER s  
5 LEFT JOIN PRODUCT p  
6 ON s.SELLER_ID = p.SELLER_ID  
7 LEFT JOIN INVENTORY i  
8 ON p.PRODUCT_ID = i.PRODUCT_ID  
9 GROUP BY s.SELLER_NAME  
10 ORDER BY s.SELLER_NAME;
```

SELLER_NAME	PRODUCT_COUNT	AVAILABLE_STOCK
Book House	1	8
Daily Mart	1	0
Fashion World	1	25
Fit Life	1	3
Gadget Point	1	5
Home Needs	1	40
Smart Home	1	18
Sport Zone	1	15
Style Hub	1	30
Tech World	1	50

10 rows selected.

Purpose:

This query helps identify how many products each seller supplies and the total stock associated with those products.

Report 2: Stock Availability Report

This report provides the current stock information for each product. It displays the product name, quantity available, and stock status.

```
SQL> SELECT p.PRODUCT_NAME,  
2         i.STOCK_QUANTITY,  
3         i.STOCK_STATUS  
4 FROM PRODUCT p  
5 JOIN INVENTORY i  
6 ON p.PRODUCT_ID = i.PRODUCT_ID  
7 ORDER BY i.STOCK_QUANTITY DESC;
```

PRODUCT_NAME	STOCK_QUANTITY	STOCK_STATUS
Smart phone	50	Available
Cotton Shirt	40	Available
Wireless Mouse	30	Available
Python programming	25	Available
Running Shoes	18	Available
Denim Jeans	15	Available
Bluetooth Speaker	8	Low Stock
Data Structures	5	Low Stock
Office Chair	3	Low Stock
Electric Kettle	0	Out of Stock

10 rows selected.

Purpose:

This report makes it easier to identify products that have sufficient stock, low stock, or no stock.

Report 3: Inventory Analysis

Inventory analysis is used to understand the overall stock condition of the products. The Week 3 requirements include total available products, out-of-stock products, highest-stocked products, and average inventory quantity.

Total Products Available

```
SQL> SELECT COUNT(*) AS TOTAL_PRODUCTS
  2   FROM INVENTORY
  3   WHERE STOCK_QUANTITY > 0;

TOTAL_PRODUCTS
-----
                9
```

This counts the products that currently have stock.

Products Out of Stock

```
SQL> SELECT COUNT(*) AS OUT_OF_STOCK_PRODUCTS
  2   FROM INVENTORY
  3   WHERE STOCK_QUANTITY = 0;

OUT_OF_STOCK_PRODUCTS
-----
                        1
```

This gives the number of products that currently have zero stock.

Highest Stocked Products

```
SQL> SELECT p.PRODUCT_NAME,
  2         i.STOCK_QUANTITY
  3   FROM PRODUCT p
  4  JOIN INVENTORY i
  5   ON p.PRODUCT_ID = i.PRODUCT_ID
  6  WHERE i.STOCK_QUANTITY = (
  7         SELECT MAX(STOCK_QUANTITY)
  8         FROM INVENTORY
  9  );

PRODUCT_NAME      STOCK_QUANTITY
-----
Smart phone              50
```

This identifies the product or products with the highest inventory quantity.

Average Inventory Quantity

```
SQL> SELECT ROUND(AVG(STOCK_QUANTITY), 2) AS AVERAGE_INVENTORY
      2 FROM INVENTORY;

AVERAGE_INVENTORY
-----
                19.4
```

This calculates the average quantity of stock maintained across the inventory records.

Combined Inventory Analysis

You can also display the main inventory statistics together using one query:

```
SQL> SELECT
  2   COUNT(CASE WHEN STOCK_QUANTITY > 0 THEN 1 END) AS TOTAL_PRODUCTS,
  3   COUNT(CASE WHEN STOCK_QUANTITY = 0 THEN 1 END) AS OUT_OF_STOCK,
  4   MAX(STOCK_QUANTITY) AS HIGHEST_STOCK,
  5   ROUND(AVG(STOCK_QUANTITY), 2) AS AVERAGE_STOCK
  6 FROM INVENTORY;

TOTAL_PRODUCTS OUT_OF_STOCK HIGHEST_STOCK AVERAGE_STOCK
-----
                9              1           50           19.4
```

Conclusion

In this week, the Seller and Inventory management section of the e-commerce database was implemented. The Seller table was created to maintain seller information, while the Inventory table was used to track the stock details of products. Relationships were established between Seller, Product, and Inventory using primary and foreign keys. Constraints were also applied to maintain valid data. Seller-wise product details, stock availability, and inventory statistics were generated using SQL joins and aggregate functions. This helped in understanding how seller information and product stock can be organized and analyzed in an e-commerce database.

